

Machine Learning Basics

AI and Economics Summer School

Andrea Ciccarone

ETH Zurich

August 31, 2026

Where This Fits

- This is the first substantive lecture of the week: everything later builds on it
- **Today (90 min)**: how predictive models are fit, regularized, tuned, and evaluated out of sample
- **Next block**: how raw text and images become vectors, and how those vectors become research variables
- **Later**: models and representations show up again inside language models, agents etc.

Outline

- 1 A (Brief) Introduction to Machine Learning
- 2 Training a Model
- 3 Flexible Models
- 4 Finding Structure: Dimension Reduction and Clustering
- 5 Wrap-up

Why do we need ML?

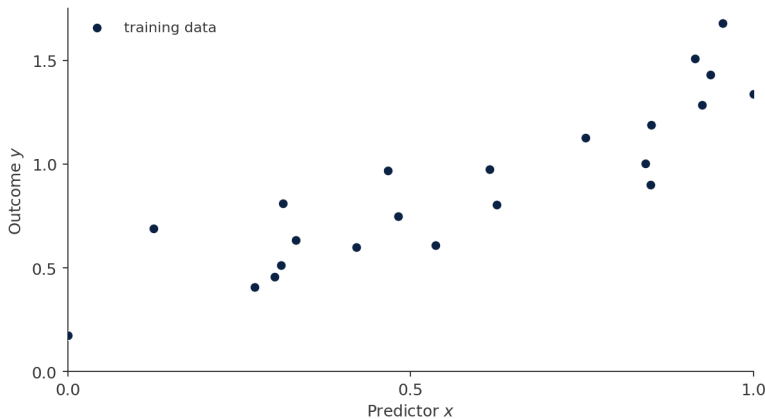
- Economists are usually after $\hat{\beta}$: the effect of X on Y (*causal identification*)
- ML is usually after $\hat{Y}$: find prediction function $\hat{f}$ from training data (Y, X) (*prediction*)
- ML is also used as a measurement device for X : it turns high dimensional data into variables you then use in a standard research design (*representation*)
- Two reasons this matters for us
 - Some economic questions *are* prediction problems
 - Some data types (text, images, videos) must first be represented numerically; that is the next block

ML: Improving from OLS?

$$Y_i = \beta X_i + \epsilon_i$$

- β_{OLS} is BLUE (best linear unbiased estimator)
 - Smallest variance among estimators with 0 bias
 - But can we improve predictions with more bias and smaller variance?
- With many parameters, OLS is unfeasible:
 - With $k > n$, $X'X$ is not even invertible
 - But even if we can find β_{OLS} , we are going to have overfitting
 - This is where ML starts coming up (Ridge, Lasso, Random Forests, Neural Networks...)
- If X_i is a text document, it will have **at least** thousands of entries (the dimension of the vocabulary)

More Flexibility Can Fit Noise Instead of Signal



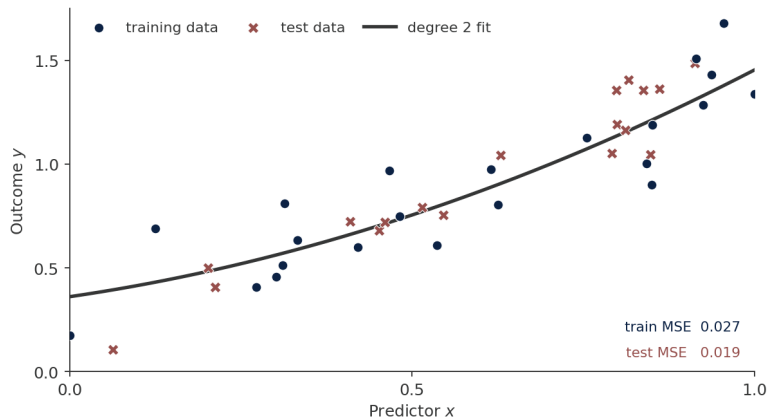
Goal: learn from the blue sample to predict outcomes for observations we have not seen.

Degree 1: a simple line captures the signal and predicts the red test sample well.



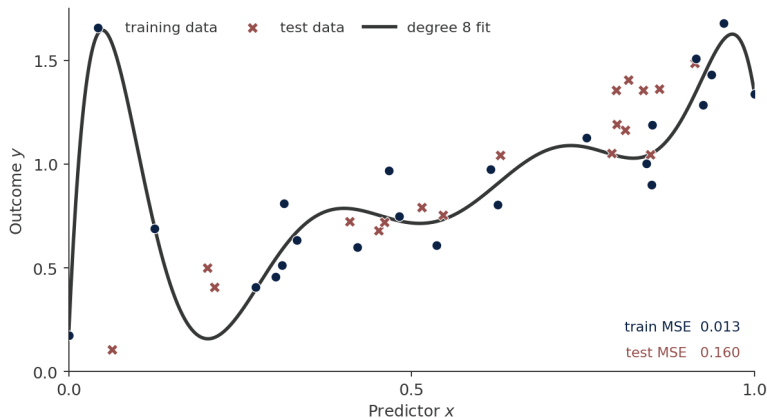
Degree 1: a simple line captures the signal and predicts the red test sample well.

More Flexibility Can Fit Noise Instead of Signal



Degree 2: training error falls, but the extra bend starts following noise; test error rises.

More Flexibility Can Fit Noise Instead of Signal



Degree 8: the curve fits training noise more closely and generalizes much worse.

Improving from OLS: Ridge

- **Ridge** estimator shrinks coefficients towards 0, reducing sensitivity of single data points
- It introduces biased, but lowers variance

$$\begin{aligned}\beta_{\text{Ridge}}(\lambda) &= \operatorname{argmin}_{\beta} \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2 \\ &= (X'X + \lambda I)^{-1} X'y\end{aligned}$$

- How do we set λ ? **Cross-validation:**
 1. Split train data into K folds
 2. Loop over values of λ
 3. Train model on $K - 1$ folds and use the K th fold for validation
 4. λ^* minimizes average prediction error across folds

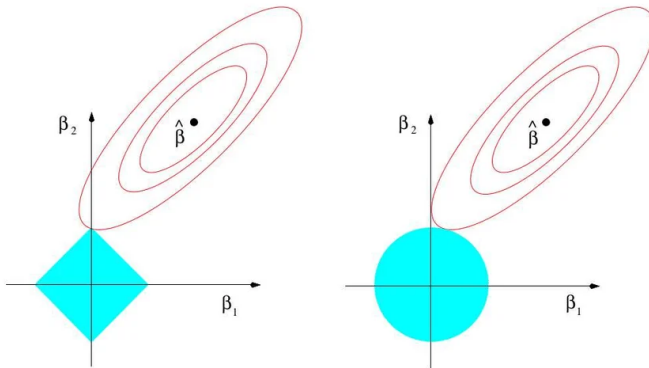
Improving from OLS: Lasso

- **Lasso** estimator also helps with overfitting but the L_1 norm shrinks some (superfluous) coefficients at 0 $\implies$ good for **feature selection**

$$\beta_{LASSO}(\lambda) = \operatorname{argmin}_{\beta} ||y - X\beta||_2^2 + \lambda ||\beta||_1$$

- Unlike Ridge, LASSO does not have a closed form solution (L1-norm component non differentiable) $\implies$ use numerical solutions via eg. LARS
- λ set, again, with cross-validation

Lasso and Ridge Geometry



- Red ellipses are RSS contours around $\hat{\beta}^{\text{OLS}}$; shaded sets are coefficient-size budgets
- The solution is where the smallest reachable contour first touches the budget set
- The ℓ_1 budget has **axis corners**, so optima often set coefficients to zero; the smooth ℓ_2 ball almost never does

Outline

- 1 A (Brief) Introduction to Machine Learning
- 2 Training a Model**
- 3 Flexible Models
- 4 Finding Structure: Dimension Reduction and Clustering
- 5 Wrap-up

$$\downarrow$$

$$x_{img}^N \in \mathbb{R}^{512}$$

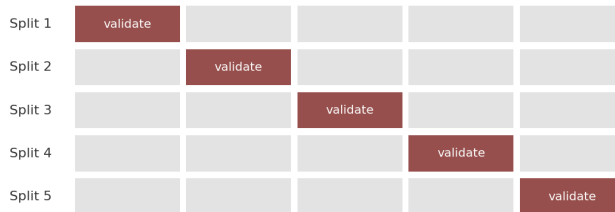
- Dataset of TV political ads, labelled by the party that paid for them
- Every frame becomes a vector, the ad is the average of its frames, and a classifier turns that into a probability
- *How* an image becomes a vector is the embeddings section. Take it as given for now

Building a Model: Train, Validate, Test



- 3 phases for building our ads model:
 1. **Training:** fit the model weights
 2. **Validation:** choose how many layers, how much dropout - regularization
 3. **Test:** touched once, to report
- **In the ads example, what is an observation?** A frame, or an ad?
- It must be the **ad**. Frames from the same ad share a set, a candidate and a colour grade. Split by frame and the model recognises the ad it already saw, and the reported accuracy is fiction

Cross-Validation



training data, split into 5 folds

- We do not care whether the model can classify ads it has already seen. We want to know whether it can identify the party of a genuinely new political ad.
- Train on $K - 1$ folds, validate on the held-out one, rotate, then average the K scores
- Every ad is validated exactly once by a model that never saw it. Stack those predictions and you get an **out-of-fold** prediction for the whole sample

What to Report

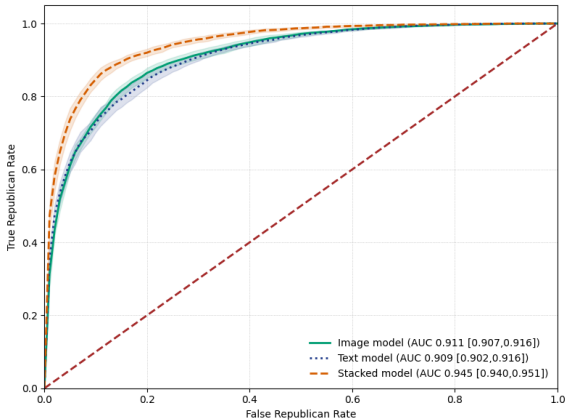
Regression: RMSE, MAE, out-of-sample R^2

Classification:

- **Accuracy** misleads when classes are unbalanced
- **Precision:** of those predicted 1, how many are 1
- **Recall:** of the true 1s, how many we caught
- **AUC:** chance a random positive outranks a random negative. Threshold-free

Ads are near 50/50, so accuracy is readable there. In most measurement tasks the positives are rare, and AUC is the honest number

The Ads Result



- Three models on the same ads: images only, transcript only, and both
- The image-only model does about as well as the text-only model
- The joint model beats both, so the two signals are **not redundant**
- Every number here is out-of-fold. That is the only reason it means anything

Logistic Regression Is Linear in the Features

For a binary outcome, logistic regression models the **log-odds** linearly:

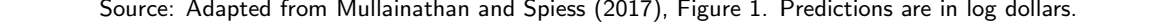
$$\log\left(\frac{\hat{p}_i}{1 - \hat{p}_i}\right) = \beta_0 + x_i'\beta \quad \Longleftrightarrow \quad \hat{p}_i = \sigma(\beta_0 + x_i'\beta).$$

- **Linear:** the log-odds and decision boundary are linear in the supplied features
- **Nonlinear probability:** the sigmoid bends the probability, not the feature relationships
- **Interactions are not automatic:** add x_1x_2 , polynomials, splines, or learned features
- **Fit:** minimize binary cross-entropy (Bernoulli maximum likelihood); ridge or lasso can regularize

In the ads model: learned features $\longrightarrow$ regularized logistic regression $\longrightarrow P(\text{Republican})$.

Outline

- 1 A (Brief) Introduction to Machine Learning
- 2 Training a Model
- 3 Flexible Models**
- 4 Finding Structure: Dimension Reduction and Clustering
- 5 Wrap-up



Regression Trees: Optimal Trees

A tree approximates $f(x)$ by a **step function**.

- Inside a region R , the prediction is just the **sample mean** of the y in it:

$$\hat{y}_R = \frac{1}{|R|} \sum_{i \in R} y_i$$

- So the only real question is where to cut. Choose the variable j and threshold c that most reduce the sum of squared residuals:

$$\min_{j, c} \sum_{i: x_{ij} < c} (y_i - \hat{y}_L)^2 + \sum_{i: x_{ij} \geq c} (y_i - \hat{y}_R)^2$$

- Then repeat inside each piece. Greedy: each split is chosen as if it were the last

Nothing is assumed about functional form. Interactions and nonlinearities are found, not specified.

Gradient Boosting in Practice

- Each new tree is fitted to the gradient of the loss left over by the ones before:

$$F_m(x) = F_{m-1}(x) + \nu f_m(x)$$

where ν is the learning rate. Small ν with many trees beats large ν with few

- XGBoost** and its relatives are the standard implementations, and on tabular data they usually beat a neural network
- Early stopping** is the regularizer that matters: add trees while a held-out score improves, stop when it does not

If your features are already structured, start here, not with deep learning.

Capturing Complex Relationships: Neural Networks

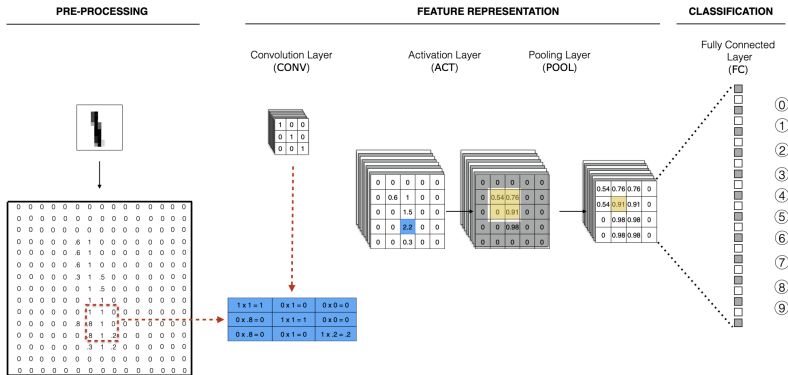
So far we have used linear models... but when working with text and particularly images we need more complex, non-linear, approaches!

- A **Neural Network** is a model:

$$g(X, \theta) = f_k(f_{k-1}(\dots f_1(X, \theta_1), \theta_2, \dots), \theta_{k-1}, \theta_k)$$

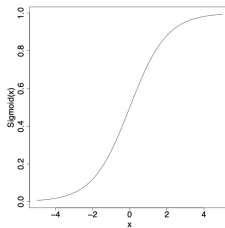
- Input X is successively transformed through a series of functions (layers) $f_1, f_2, \dots, f_k$
- Each θ_i is a vector or matrix of weights for layer i (this is the **learning** part)
- A *typical* f_i may look like $f_i(X, \theta_i) = \sigma(\theta_i X + b_i)$, where $\sigma(x)$ is often $\tanh(x)$, $\text{logit}(x)$ or $\max(0, x)$
- Output layer can be a simple linear transformation $f_k(X, \theta_k) = \theta_k X + b_k$ (for continuous outcomes) or a softmax (for discrete classification)
- Our goal is to find the parameters θ that minimize some loss function $\mathcal{L}$

Picture of a (Convolutional) Neural Network

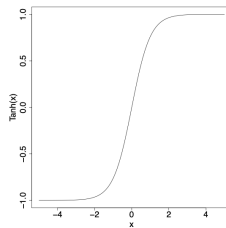


Source: Torres and Cantù (2022)

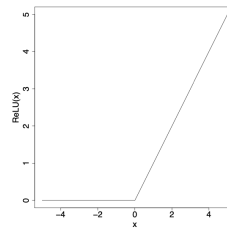
Examples of Activation Functions σ



(a) $Sigmoid(x) = \frac{1}{1+e^{-x}}$



(b) $Tanh(x) = \frac{2}{1+e^{-2x}}$



(c) $ReLU(x) = \begin{cases} 0 & \text{if } x < 0, \\ x & \text{otherwise.} \end{cases}$

Source: Torres and Cantù (2018)

How to Estimate a Neural Network

Forward Pass:

1. Given a starting θ , NN takes input X and sequentially applies each function f_i : $a_1 = f_1(X, \theta_1)$,
 $a_2 = f_2(a_1, \theta_2), \dots g(x, \theta) = a_k = f_k(a_{k-1}, \theta_k)$
2. Once we have $g(X, \theta)$, we can compute the loss function $\mathcal{L}(g(x, \theta), Y)$

Backpropagation:

1. From the last layer, chain rule to compute:

$$\frac{\delta \mathcal{L}}{\delta \theta_k} = \frac{\delta \mathcal{L}}{\delta a_k} \cdot \frac{\delta a_k}{\delta \theta_k}$$

and (recall a_{k-1} is the input of the final layer)

$$\frac{\delta \mathcal{L}}{\delta a_{k-1}} = \frac{\delta \mathcal{L}}{\delta a_k} \cdot \frac{\delta a_k}{\delta a_{k-1}}$$

2. We can then *backpropagate* the gradients through each layer, eg.:

$$\frac{\delta \mathcal{L}}{\delta \theta_{k-1}} = \frac{\delta \mathcal{L}}{\delta a_{k-1}} \cdot \frac{\delta a_{k-1}}{\delta \theta_{k-1}}$$

Gradient Descent

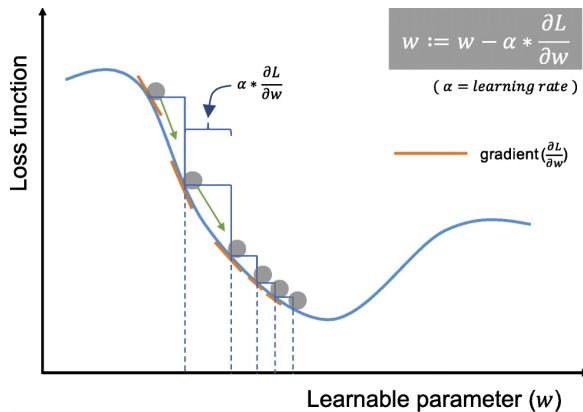
- With backpropagation, we can compute each $\frac{\delta \mathcal{L}}{\delta \theta_i}$
- We update parameters θ by moving on the opposite direction of the gradient $\nabla_{\theta} \mathcal{L}$, because this is the direction of steepest descent
- At each step of the algorithm we update the parameters so that

$$\theta^{t+1} = \theta^t - \alpha \nabla_{\theta} \mathcal{L}$$

where α is called the *learning rate*

- We iterate the algorithm until loss is lower than some threshold

Gradient Descent



Source: Yamashita et al. (2018)

Outline

- 1 A (Brief) Introduction to Machine Learning
- 2 Training a Model
- 3 Flexible Models
- 4 Finding Structure: Dimension Reduction and Clustering**
- 5 Wrap-up

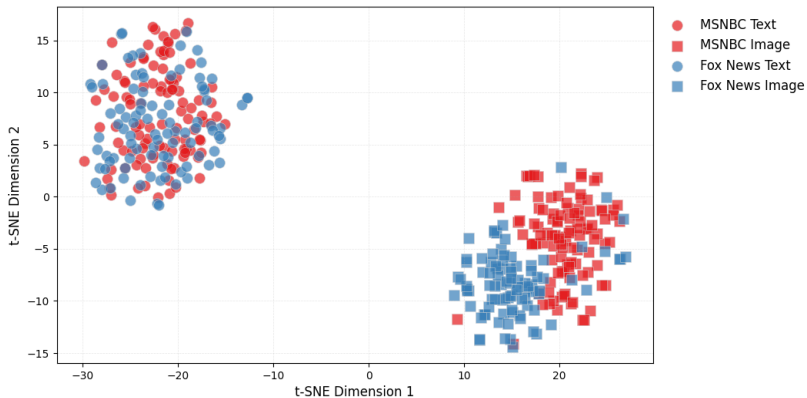
Unsupervised Learning: Finding Structure in X

- In supervised learning we observe a target Y and learn $\hat{f}(X)$
- In unsupervised learning we observe only $X = (x_1, \dots, x_N)'$, with $x_i \in \mathbb{R}^p$
- We look for a lower-dimensional representation of the variation in X , or groups of similar observations
- **PCA**: replace p variables with a small number of continuous components
- **k-means**: replace N observations with membership in K groups
- For now, take the numerical vectors x_i as given. The next block explains how text and images become vectors

Unsupervised Tasks: PCA

- PCA finds a lower-dimensional representation that retains as much of the variance as possible
- Find projection function $f(\lambda) = \mu + V_q \lambda$ where V_q maps p (target dimension) into q (original dimension)
- Solve $\min_{\lambda, V_q, \mu} \sum_{i=1}^N \|x_i - \mu - V_q \lambda_i\|^2$
- To perform this, perform Singular Value Decomposition on $X \implies X = UDV^T$
- The columns of V contain the eigenvectors of $X^T X$ and give the directions in the original feature space - ordered by explained variance
- The first component is the direction capturing the the most variance so if we project the data onto this direction we retain the largest amount of variability
- PCA is the simplest way to perform dimension reduction. Other sophisticated alternatives exist (eg. t-SNE)

Component Reduction: Example



Unsupervised Tasks: k-means clustering

- Another useful technique is k-means clustering, used to group observations into clusters
- Within each cluster, points are as similar as possible to each other
- We solve:

$$\min_S \sum_{j=1}^K \sum_{i \in C_j} ||x_i - S_j||^2$$

- We are minimizing within-cluster variance (K is the number of clusters), where S_j is the cluster center
- So we are essentially finding the cluster centers, which also provides representative observations for each cluster

Outline

- 1 A (Brief) Introduction to Machine Learning
- 2 Training a Model
- 3 Flexible Models
- 4 Finding Structure: Dimension Reduction and Clustering
- 5 **Wrap-up**

Recap

Core logic

1. Use a flexible functional form
2. Limit its expressiveness through regularization
3. Tune how much to regularize using validation data

Important choices

- Loss function
- Data split
- Function class
- Regularizer

Implementation: Training the Ads Classifier

Predict whether an ad is Republican from its CLIP embedding:

```
X_train, X_test, y_train, y_test = train_test_split(
    ad_embeddings, party, test_size=.20,
    stratify=party, random_state=44)

model = keras.Sequential([
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dropout(.20),
    keras.layers.Dense(1, activation="sigmoid")
])

model.compile(optimizer="adam",
              loss="binary_crossentropy",
              metrics=[keras.metrics.AUC(name="auc")])

model.fit(X_train, y_train,
          validation_split=.20, epochs=50,
          callbacks=[keras.callbacks.EarlyStopping(
              monitor="val_auc", mode="max", patience=4,
              restore_best_weights=True)])

test_auc = model.evaluate(X_test, y_test)[1]
```

Appendix: Why Cross-Validation Can Lie to You

- **Leakage through preprocessing:** if you standardize, select features, or fit a vectorizer on the full sample *before* splitting, the validation score is contaminated. Fit inside the fold
- **Grouped data:** repeated observations of the same firm, judge, or channel must go in the same fold
- **Time:** with any temporal structure, validate forward, never on the past
- **Hyperparameter search:** if you cross-validate hundreds of configurations and report the best, that number is optimistically biased. Keep a genuine test set